

UNIFAAT

2026.2

07 - Introdução ao Python

Prof. Luan Lourenço

Chave SSH: um par por máquina

Cada ambiente gera o seu próprio par de chaves. O WSL guarda o dele em `~/.ssh/` e o Windows em `C:\Users\NOME\.ssh\`: no mesmo notebook, são **dois pares diferentes**, um pro Linux do WSL e outro pro Windows — ambientes isolados, como na Aula 02.

```
# máquina pessoal  
chave pública da conta  
foto de perfil → Settings → SSH and GPG keys  
acesso a todos os seus repositórios
```

```
# máquina da faculdade  
chave só do repositório  
repositório → Settings → Deploy keys  
marcar "Allow write access" pra poder dar push
```

! Uma chave pública só pode ser cadastrada **uma vez em todo o GitHub**: se ela já está numa conta ou em outro repositório, o GitHub recusa com `Key is already in use`. Na faculdade, cada repositório precisa da sua própria chave.

Python é o **meio**, não o **fim**

O que importa no curso, e o que vai junto pra qualquer linguagem, são quatro coisas:

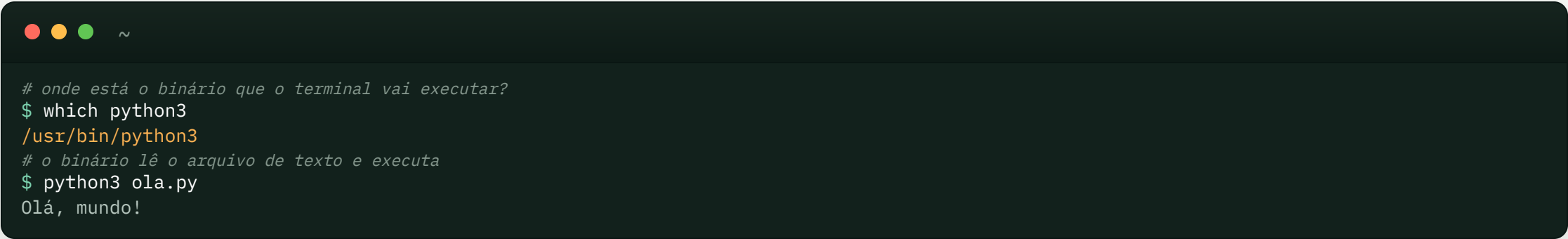
```
# o fim  
algoritmos  
estruturas de dados  
orientação a objetos  
boas práticas
```

```
# o meio  
Python, neste ano  
nas próximas disciplinas a linguagem muda:  
Desenvolvimento Web e Frontend usam JavaScript e TypeScript
```

A linguagem é só a ferramenta. Quem domina os quatro conceitos troca de linguagem aprendendo apenas a sintaxe nova.

Python é um **binário** que lê **texto**

Como `ls` e `git`, `python3` é um binário (escrito em C). Ele lê um arquivo `.py`, que é texto puro, e executa: por isso o Python é uma linguagem **interpretada**.

A terminal window with a dark background and light-colored text. The window has three colored window control buttons (red, yellow, green) and a tilde symbol (~) in the title bar. The text inside the terminal shows a series of commands and their outputs, including a comment about the binary path and the execution of a script named ola.py.

```
# onde está o binário que o terminal vai executar?  
$ which python3  
/usr/bin/python3  
# o binário lê o arquivo de texto e executa  
$ python3 ola.py  
Olá, mundo!
```

Por que `python3` e não `python`? Em 2008 o Python 3 quebrou a compatibilidade com o 2, e as duas versões conviveram até 2020 — cada uma com o seu binário.

Instalando no WSL e no Windows

Ambientes isolados (Aula 02): **cada lado precisa do seu binário**. Dá pra usar Python nos dois.

● ● ● Ubuntu (WSL)

```
# o Ubuntu já traz o python3
$ python3 --version
# faltam o pip e o venv
$ sudo apt update
$ sudo apt install python3-pip python3-venv
```

● ● ● PowerShell (Windows)

```
# instalador do python.org,
# marcando "Add python.exe to PATH"
PS> python --version
PS> where.exe python
```

Download: <https://www.python.org/downloads/>

! No Windows, digitou `python` e abriu a Microsoft Store? O Python ainda não está instalado, ou a caixa do PATH ficou desmarcada.

Dependências, **venv** e Git

- **Dependência:** biblioteca de terceiros, instalada com `pip` a partir do PyPI
- Projetos pedem versões diferentes da mesma biblioteca; o Ubuntu nem deixa instalar no Python do sistema (`externally-managed-environment`)
- **venv:** uma pasta com um Python só do projeto. `.venv/` vai no **.gitignore**; `requirements.txt` vai pro **repositório**

```
meu-projeto/  
  
# cria o ambiente na pasta oculta .venv  
$ python3 -m venv .venv  
# ativa (Windows: .venv\Scripts\Activate.ps1)  
$ source .venv/bin/activate  
# instala só dentro deste projeto  
(.venv) $ pip install requests  
# lista as bibliotecas e versões / recria em outra máquina  
(.venv) $ pip freeze > requirements.txt  
(.venv) $ pip install -r requirements.txt
```

Tipos básicos

No Python, o tipo pertence ao valor, não à variável: não se declara nada, e `type()` mostra qual é.

python3

```
>>> type(45), type(7.5), type(True), type("Ana")
(<class 'int'>, <class 'float'>, <class 'bool'>, <class 'str'>)
>>> 2 ** 100 # int não para no 255
1267650600228229401496703205376
>>> 0.1 + 0.2 # float: imprecisão da representação binária
0.30000000000000004
>>> True + True # bool: True vale 1, False vale 0
2
>>> ord("A"), chr(65) # str: cada caractere é um número (ASCII, Aula 03)
(65, 'A')
>>> 45 // 2, 45 % 2 # quociente e resto
(22, 1)
```

Guardando **mais de um valor**

python3

```
# list: o "array" do Python – ordenada, índice desde 0, mutável
>>> notas = [7, 3, 9]; notas.append(5); notas[0]
7
# tuple: imutável, valores que andam juntos (e desempacotam)
>>> quociente, resto = divmod(45, 2)
>>> a, b = 1, 2; a, b = b, a; a, b
(2, 1)
# dict: busca por chave
>>> ascii_parcial = {"A": 65, "a": 97, "0": 48}
>>> ascii_parcial["A"]
65
```

! `b = a` com listas **não copia**: cria um segundo nome pro mesmo objeto. Pra copiar: `a.copy()`.

Algoritmo: passos finitos e sem dúvida

A palavra vem de **al-Khwarizmi**, matemático persa do século IX. Algoritmos são bem mais velhos que computadores — e vocês já executaram um no quadro:

1. divide por 2, anota o resto
 2. troca o número pelo quociente
 3. repete até o quociente ser 0
 4. lê os restos de baixo pra cima

n	n % 2	n // 2	binário
45	1	22	1
22	0	11	01
11	1	5	101
5	1	2	1101
2	0	1	01101
1	1	0	101101

O mesmo 45 da revisão (Aula 06). Agora, em código.

Decimal → binário

```
binario.py

def decimal_para_binario(numero):
    # caso de borda: no quadro ninguém converte o zero
    if numero == 0:
        return "0"

    binario = ""
    while numero > 0:
        resto = numero % 2
        # resto novo vai NA FRENTE: é o "ler de baixo pra cima"
        binario = str(resto) + binario
        numero = numero // 2

    return binario

print(decimal_para_binario(45)) # 101101
print(decimal_para_binario(65)) # 1000001 -> o "A"
```

Trocar pra `binario + str(resto)` inverte o resultado – o mesmo erro do quadro, agora em código.

Testando os 256 valores de um byte

Testar alguns números à mão é um começo. Um engenheiro compara com uma referência confiável — em muitos casos de uma vez. O `bin()` do Python é a referência; o `assert` para tudo se a comparação falhar.

```
binario.py

print(bin(45))  # 0b101101 (o "0b" é só o prefixo)
print(int("101101", 2))  # 45 -> o caminho de volta

for n in range(256):
    assert decimal_para_binario(n) == bin(n)[2:]

print("Os 256 valores de um byte conferem!")
```

Três linhas conferem o byte inteiro. É o embrião dos testes automatizados.

Bubble sort: compara vizinhos e troca

ordenacao.py

```
def bubble_sort(valores):
    lista = valores.copy()  # não mexe na lista de quem chamou
    n = len(lista)

    for passada in range(n - 1):
        trocou = False
        for i in range(n - 1 - passada):
            if lista[i] >= lista[i + 1]:
                continue  # já em ordem: sem else
            lista[i], lista[i + 1] = lista[i + 1], lista[i]  # troca via tupla
            trocou = True

        if not trocou:
            break  # passada sem troca: pronto

    return lista
```

Passada a passada

passada	lista no fim da passada	o que aconteceu
início	[7, 3, 9, 1, 5]	–
1	[3, 7, 1, 5, 9]	o 9 chegou ao fim
2	[3, 1, 5, 7, 9]	o 7 chegou ao lugar
3	[1, 3, 5, 7, 9]	3 e 1 trocaram
4	[1, 3, 5, 7, 9]	nenhuma troca: para

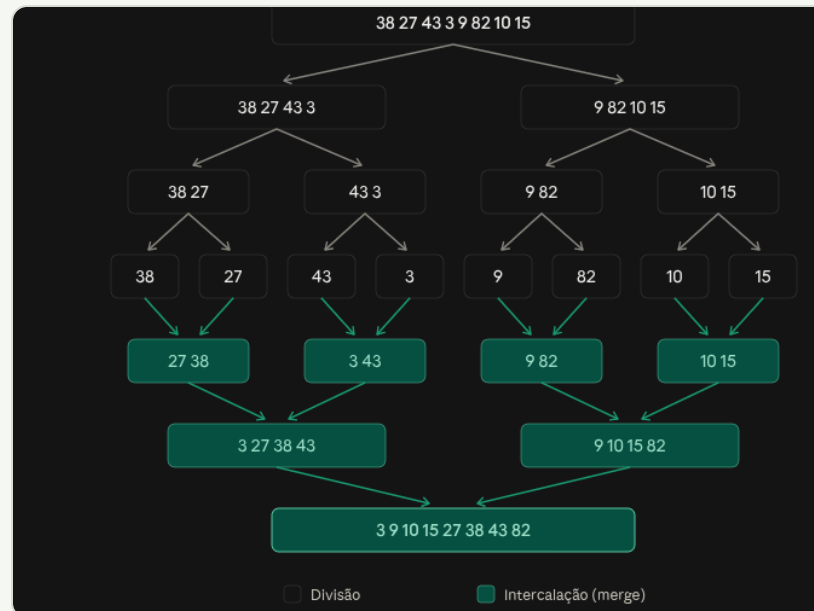
python3

```
# só usa <= ... então ordena strings também
>>> bubble_sort(["Carla", "ana", "Bruno"])
['Bruno', 'Carla', 'ana']
```

"ana" foi pro fim: pro computador, "a" é 97 e "C" é 67. A tabela da Aula 03, de novo.

Merge sort: **dividir por 2** até não dar mais

- **Dividir:** corta a lista ao meio, e cada metade ao meio de novo, até sobrar 1 item (1 item já está ordenado)
- **Conquistar:** junta (intercala) as metades de volta, sempre em ordem



8 itens → 3 divisões por 2 ($2^3 = 8$). A mesma divisão por 2 do binário, agora aplicada a uma lista.

Merge sort em Python

```
ordenacao.py

def merge_sort(lista):
    if len(lista) <= 1: # caso base: 1 item já está ordenado
        return lista

    meio = len(lista) // 2 # dividir por 2
    esquerda = merge_sort(lista[:meio]) # a função chama a si mesma
    direita = merge_sort(lista[meio:])
    return intercalar(esquerda, direita)

def intercalar(esquerda, direita):
    resultado = []
    i = 0
    j = 0
    while i < len(esquerda) and j < len(direita):
        if esquerda[i] <= direita[j]:
            resultado.append(esquerda[i])
            i += 1
            continue # sem else
        resultado.append(direita[j])
        j += 1
    resultado.extend(esquerda[i:]) # o que sobrou de um dos lados
    resultado.extend(direita[j:])
    return resultado
```

Uma função que chama a si mesma em partes menores é **recursiva** – o mesmo conceito do `rm -r` da Aula 03.

Por que dividir por 2 importa

Quantas vezes dá pra dividir 1.000 por 2 até chegar em 1? Umas **10** — a mesma quantidade de dígitos de 1.000 em binário (`1111101000`). O merge sort faz ~10 rodadas de intercalação; o bubble sort compara quase todo mundo com todo mundo.

itens	bubble sort (pior caso)	merge sort
1.000	~500 mil comparações	~10 mil
1.000.000	~500 bilhões	~20 milhões

O `sorted()` do Python usa o Timsort, que é construído em cima do merge sort. Dividir por 2 vai voltar muito nas próximas aulas.

tarefa

TF 07: quick sort

Num repositório novo `tf-07`, com `venv` e `.gitignore`, implementar num `ordenacao.py`:

- `quick_sort(valores)`: escolhe um item como **pivô** (por exemplo, o do meio), separa em menores, iguais e maiores que ele, e ordena os menores e os maiores chamando a si mesma
- Resultado: menores ordenados + iguais + maiores ordenados
- Testar imprimindo com uma lista fora de ordem: `print(quick_sort([7, 3, 9, 1, 5]))`
- Sem `else`, sem `sorted()` / `.sort()`, comentários com as próprias palavras, 2+ commits, `.venv` fora do repositório

Entrega: link do repositório público no GitHub, pelo formulário abaixo — docs.google.com/forms/d/e/1FAIpQLSdhAbpphs3ip17-2EHyuzWIsQKm24xJ_siqw3VGYNdNIuRPw/viewform

Repositório base da disciplina (o `src/` visto em aula): <https://github.com/luan-tavares/unifaat-es-2026>

Prazo: **14/10, até 23:59.**